

FLEX

Content-adaptive early exit in a learned image decoder

The bitstream does not change. The decoder decides how much of itself to run.

454 GMAC → 324 GMAC per 1080p frame

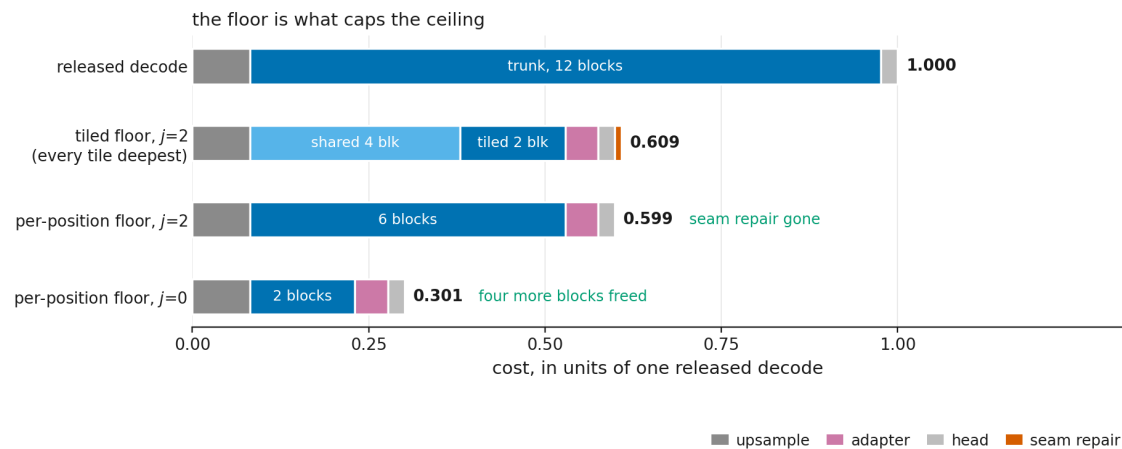
$\Delta\text{PSNR} \leq 0.1$ dB on every frame · 264 of 265 · no weight retrained

The decoder spends the same arithmetic everywhere

motivation

Q1 How should we motivate the work?

A learned decoder runs a fixed network over the whole frame, and the network is fixed by design — that is what lets any decoder read any file. A flat sky and a face are decoded at the same price. Almost every attempt to make decoders cheaper changes the model, and so the bitstream: a file encoded yesterday cannot benefit.



What is left to vary is not what the decoder is, but how much of it runs where.

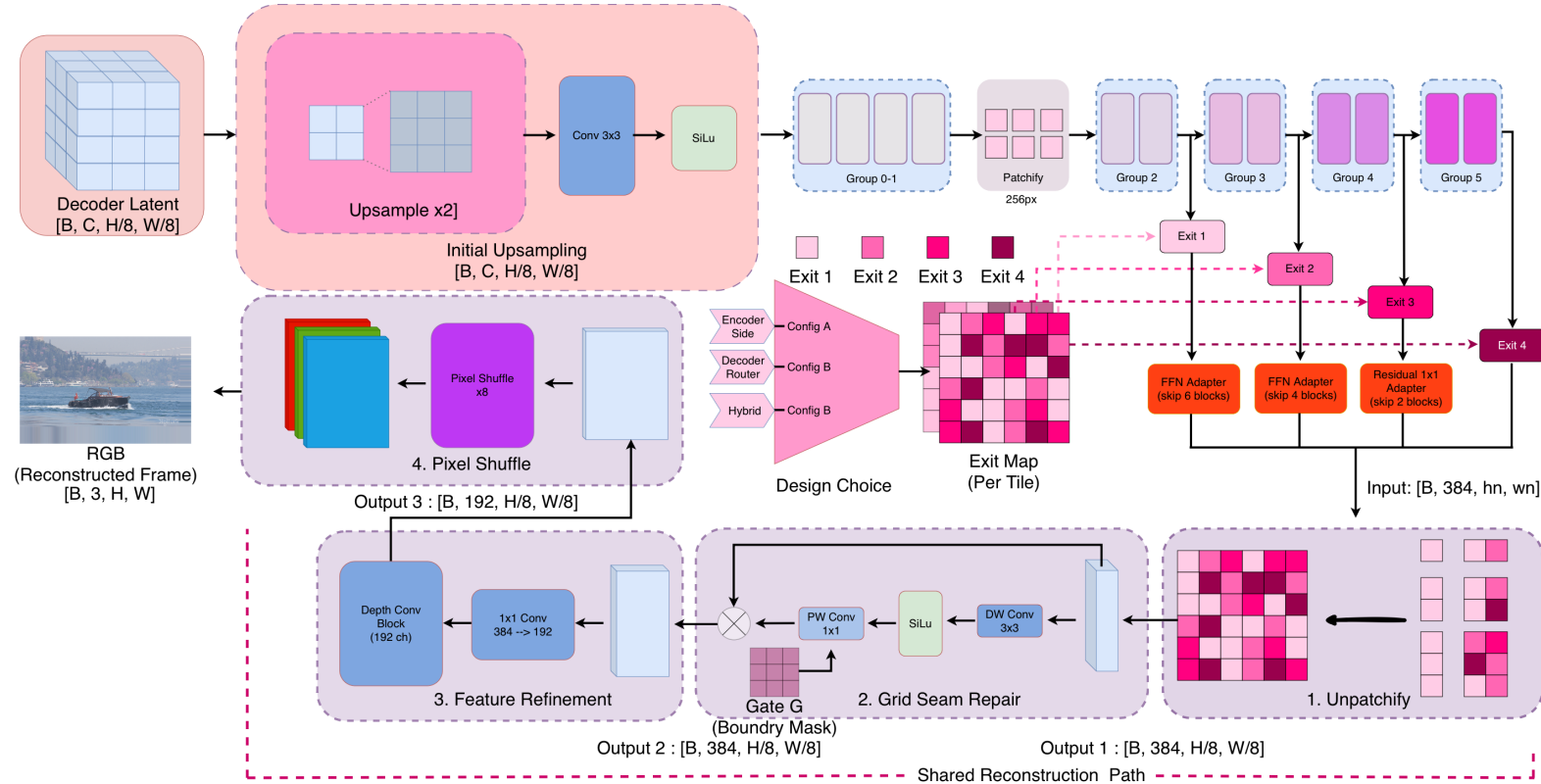
The trunk is 89.4% of the decode and it is the only routable part. The opening upsample (8.2%) and the head (2.4%) run full-frame whatever we do, which is why no allocation can save more than about 89%.

So the question is a scheduling question, not an architecture question, and it can be asked without touching the file.

The method: exits on a shared trunk, one per tile

method

Q1 What exactly is fixed, and what is free?



Two groups run for every tile; **patchify** then cuts the feature map into 256 px tiles and each tile leaves the trunk at its own depth. An adapter reconciles the early exit with a head fitted to the full one; the deepest exit carries no adapter and is bit-exact with the released decoder. The encoder, the entropy model and the coded latent are untouched.

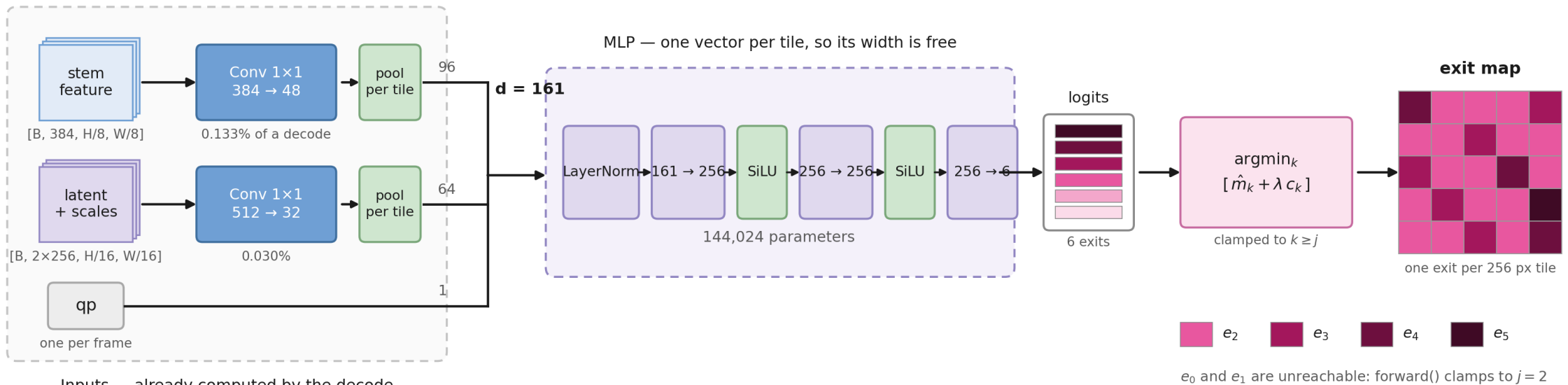
The router: one exit per tile, from tensors the decode holds

method

Q4 Which experiments are needed? — the decoder-side decision

FLEX router

one exit per tile, from tensors the decode already holds



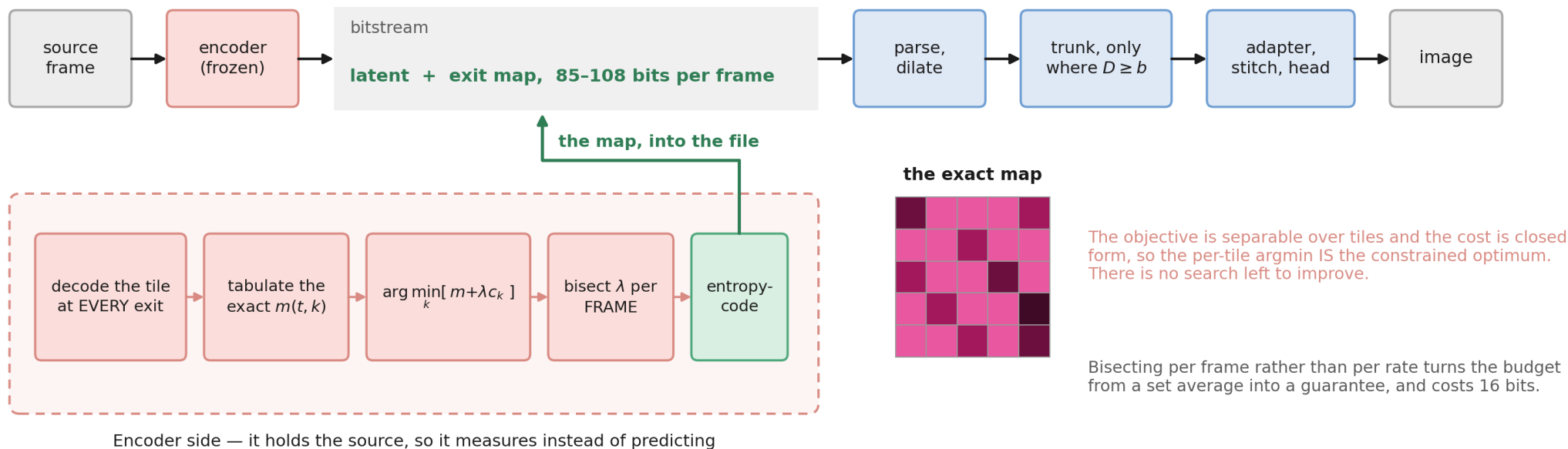
Trained by regret-weighted cross-entropy to the oracle's own choice: the weight is what the mistake costs, so capacity is spent where it matters.

Agreement with the oracle 78.7%, against 37.9% for the best constant choice; 76.2% held out. Total cost 0.163% of one decode.

The entropy model's scales are what v1 threw away, and the two signals most correlated with the oracle's choice live there.

A signalled

the encoder measures the allocation, minimises it exactly, and sends the answer



mean MAC saving, 0.1 dB

28.32%

frames inside the budget

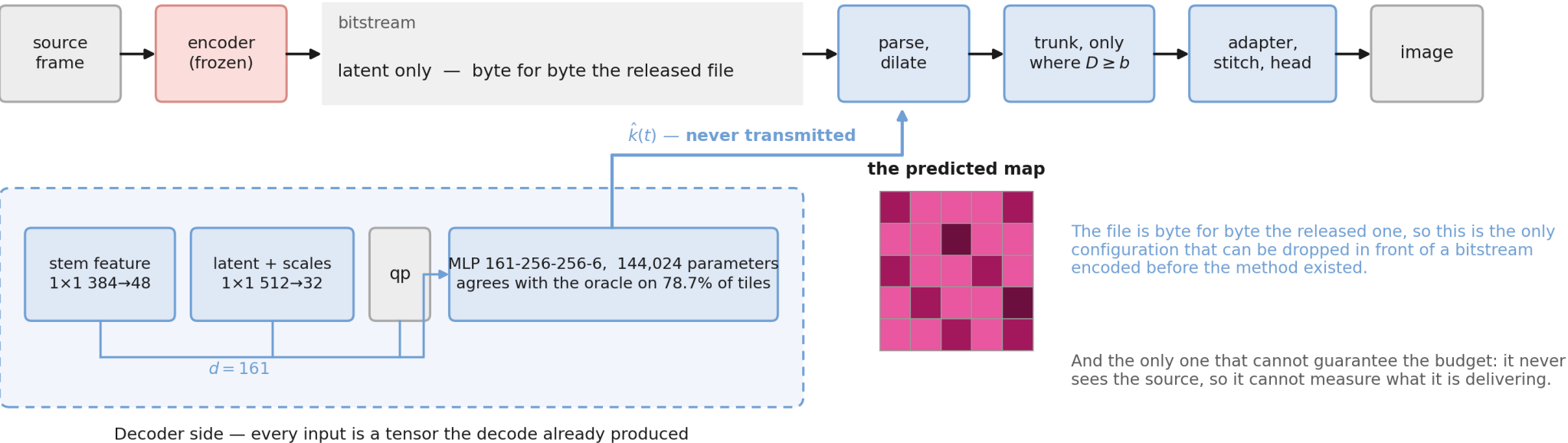
264 / 265

added to the bitstream

108 bits, about 2×10^{-4} of the frame

B bitstream-identical

nothing is added to the file; the decoder recovers the map from what it already holds



mean MAC saving, 0.1 dB

24.69%

added to the bitstream

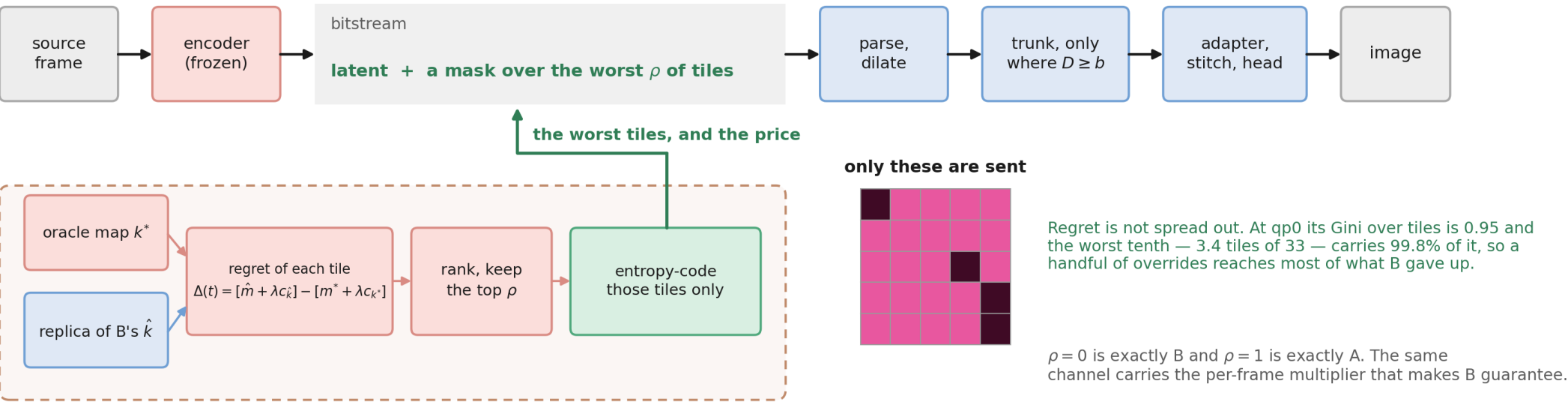
0 bits

router cost

0.163% of one decode

C partial signalling

the map is sent only where the predictor is worst, and the price is sent always



Both sides — the encoder runs a REPLICA of the decoder's predictor

at $\rho = 0.1$, qp0

33.72%

map bits at that point

26 bits/frame

tiles overridden

3.4 of 33

Main result: 454 GMAC → 324 GMAC, with a per-frame guarantee

result

Q3 What is the main message?

rate	bpp	released PSNR	ours	Δ PSNR	decoder GMAC	saved
qp 0	0.2313	32.536 dB	32.447 dB	−0.089	299	34.11%
qp 16	0.2673	35.335 dB	35.241 dB	−0.094	312	31.30%
qp 32	0.3351	38.135 dB	38.041 dB	−0.095	329	27.56%
qp 48	0.4559	40.777 dB	40.683 dB	−0.093	337	25.74%
qp 63	0.6716	43.155 dB	43.060 dB	−0.096	341	25.00%
mean					324	28.74%

Same bitstream, byte for byte. Same encoder, same entropy model. The only new weights are five pointwise adapters, and their last linear map was solved in closed form rather than trained.

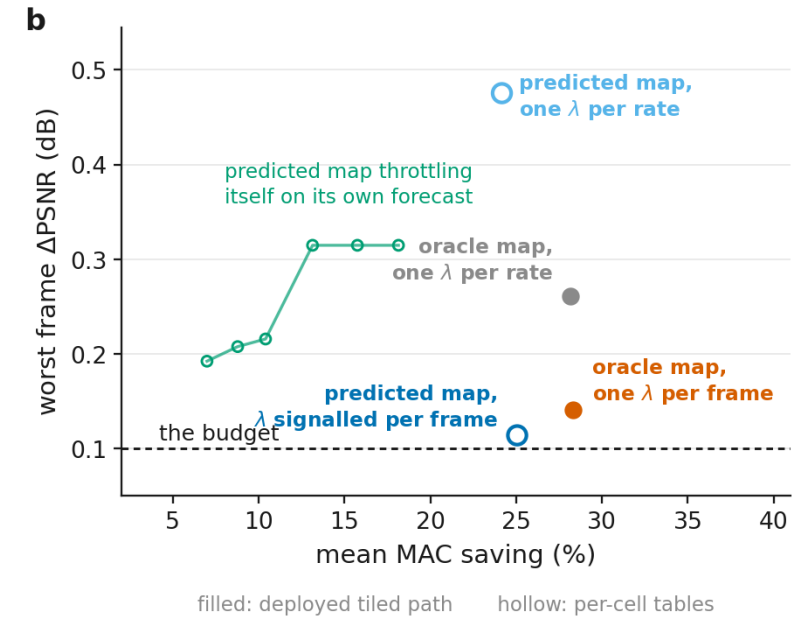
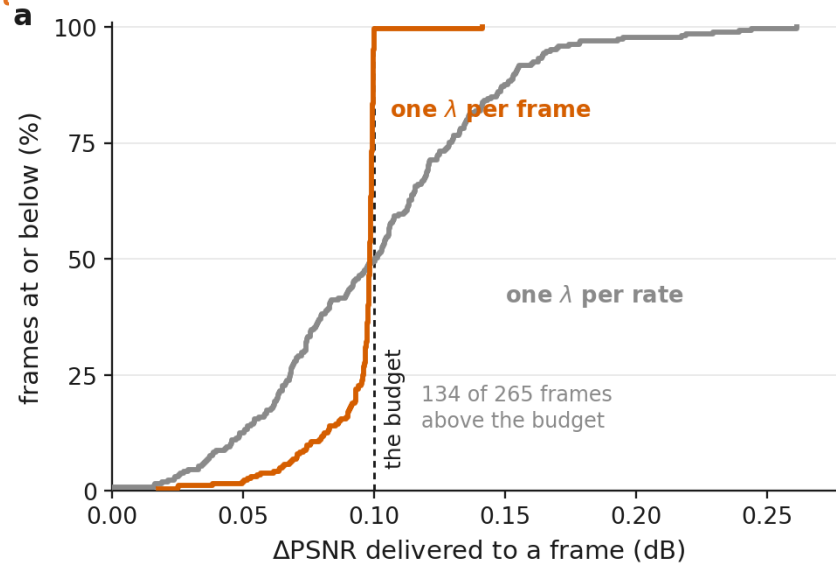
264 of 265 frame-rate pairs are served at or under 0.1 dB. The one exception is a frame whose tiling floor is 0.141 dB — above the budget before any tile has exited early, so no allocation can rescue it.

Signalling cost: 85 to 108 bits per frame, including 16 bits for the multiplier. About 2×10^{−4} of the frame it steers.

A budget the frame keeps, not one the set keeps on average

result

Q3 The message is a guarantee, not an average



where the multiplier is chosen	mean saving	frames over budget	95th pct	worst frame
one per rate (the usual protocol)	28.15%	134 / 265	0.167 dB	0.261 dB
one per frame	28.32%	1 / 265	0.100 dB	0.141 dB

The guarantee is not paid for. A single multiplier solves a Lagrangian on *pooled* squared error, while the constraint is on a mean of logarithms — so the set-average form is not optimal even for the constraint it states.

Q2 What baselines should we use?

baseline	mean saving at 0.1 dB	of the oracle
oracle — exhaustive search, upper bound	33.32%	100.0%
rate-rank — one number per tile, no parameters	30.03%	90.1%
gradient boosting on 26 hand-made features	27.00%	81.0%
ridge on the same features	30.61%	91.9%
flat — one curve for every tile, no adaptivity	15.10%	45.3%

The comparison that matters is not against another network. It is against **the exhaustive search**, which bounds what any router can reach, and against **a rule with no parameters** that reads how many bits the entropy coder already spent on a tile.

The free rule beats the learned head. We report that rather than hide it: the learned router exists because it can be conditioned on the operating point, not because it predicts better.

Q5 Which ablation studies? — the decode geometry

variant	mean saving	against the row above
tiled 256 px (the paper)	27.64%	
per-position, 256 px cells	31.51%	+3.87
per-position, 64 px cells	33.27%	+1.77

cell size	mean saving	band cost
256 px	31.51%	0.33 pts
128 px	32.66%	0.73 pts
64 px	33.27%	1.56 pts
32 px	33.59%	2.75 pts

Cutting the frame into tiles costs quality before any tile has exited early — the **floor**. Per-position decoding removes it; finer cells then buy more, until the **band** (positions still computing to serve a deeper neighbour) eats the gain.

Both rows are on the same checkpoint with no weight retrained.

Ablation: the gain is where the adapters are fitted, not that they are

ablation

Q5 Which ablation studies? — the refit and its controls

arm	mean saving	against baseline
baseline, epoch 9	28.15%	—
survivors, matched lambda	28.80%	+0.64
all tiles (control)	28.34%	+0.19
survivors, WRONG lambda (control)	28.14%	-0.01

Each adapter's last linear map is re-solved in closed form — a ridge regression onto the deepest exit's feature — on **the tiles that actually reach that exit**, at a multiplier matched to the deployed usage. 443,520 parameters, 0.98% of the decoder, two minutes on one card, no gradient step.

Fitting on all tiles instead gives +0.19 rather than +0.64, so two thirds of the gain is the conditioning. Fitting on survivors chosen at the *wrong* multiplier gives −0.01: same method, same data, wrong conditional, and the gain disappears entirely.

Q5 Which ablation studies? — the negative result that explains the positive one

arm	mean saving	against baseline
baseline, no fine-tune	28.32%	+0.00
6000 steps, uniform weights (control)	28.32%	-0.01
6000 steps, usage-matched weights	28.32%	-0.01
closed-form refit, no training	28.74%	+0.42

Six thousand steps with the anchor on the released decoder and the adapters at the main run's own learning rate return exactly the baseline — with the usage prior and without it.

The adapters are not undertrained. They have converged, for an **average** distribution, and stochastic gradient descent on OpenImages cannot be aimed away from it: every step sees every tile, and the multiplier matching that makes the closed form work has no analogue in the training loop. The closed form can state the conditional directly. That is the whole of the difference.

Q4 Which experiments? — the allocation itself

rate	e0	e1	e2	e3	e4	e5	mean exit	trunk blocks run
qp 0	0.0%	0.0%	80.2%	14.1%	4.7%	1.0%	2.26	6.5 / 12
qp 16	0.0%	0.0%	69.6%	20.5%	7.8%	2.2%	2.42	6.8 / 12
qp 32	0.0%	0.0%	56.8%	25.0%	11.8%	6.4%	2.68	7.4 / 12
qp 48	0.0%	0.0%	50.4%	25.0%	16.6%	7.9%	2.82	7.6 / 12
qp 63	0.0%	0.0%	47.4%	23.8%	19.2%	9.6%	2.91	7.8 / 12
all	0.0%	0.0%	60.9%	21.7%	12.0%	5.4%	2.62	7.2 / 12

e0 and e1 are exactly zero — measured, not assumed. Under the clamp the first two exits are unreachable, and they are also not cheaper: the first four blocks run full-frame before patchify, so e0, e1 and e2 all cost 0.5716 of a decode.

Two exits carry 82.6% of the tiles. That is why the closed-form refit concentrates its gain on them.

Q6 What are the limitations?

	mean saving	images inside 0.1 dB	mean tiling floor
Kodak, qp 0	33.93%	24 / 24	0.0220 dB
Kodak, qp 32	25.83%	24 / 24	0.0345 dB
Kodak, qp 63	5.87%	16 / 24	0.0894 dB

1. **The floor can exceed the budget.** On Kodak at the top rate, 8 of 24 images cannot reach 0.1 dB at all: cutting them into tiles already costs more than the budget allows. It is not an allocation failure and no granularity fixes it.

2. **Saving is counted in multiply-accumulates.** A real block-sparse kernel keeps most of it at 256 px cells and much less at 64 px — the finer allocation is worth less once implementability is charged, which inverts the ideal ranking.

block-sparse kernel	256 px cells	64 px
ideal (per position)	31.51%	33
B = 2 feature positions	31.28%	32
B = 4 feature positions	30.88%	30
B = 8 feature positions	29.94%	27
B = 16 feature positions	27.91%	21
B = 32 feature positions	24.17%	8

3. **One decoder, intra only.** The inter path is untouched and no end-to-end GOP number is claimed.

Q6 Possible directions for future work

1. Three more decoders, not one

Every number here is DCVC-UF. The mechanism assumes only a residual trunk with a shared head, so it should port — but a claim about learned decoders needs more than one of them. Candidates with released weights and the same shape of synthesis transform: ELIC, MLIC++, and the TCM/Mixed-Transformer family. Each needs the ladder attached, the adapters warm-started to identity, and the cost table re-measured with hooks; the router and the allocation transfer unchanged.

2. Per-position decoding, and the kernel that makes it real

It removes the floor, which is what makes the guarantee unreachable on hard content, and is worth +2.3 points at 256 px cells once a 32 px block-sparse kernel is charged. The missing piece is the kernel and an end-to-end wall-clock measurement.

3. A budget chosen per rate rather than one for all

Each rate exhausts the ladder at its own tolerance. Allocating one quality budget across rates is a second Lagrangian and costs nothing to solve.

4. The training objective the deployment actually uses

The closed form showed the gain is in the conditioning. A training loop that samples tiles from the deployed exit distribution — not just weights the loss — is the version of Tier B that was not tried.

	mean saving	guarantee	added bits	training
A signalled	28.74%	every frame	85–108 / frame	none
B bitstream-identical	24.72%	distribution only	0	none
C partial signalling	between the two	every frame	26 at $\rho = 0.1$	none

454 GMAC → **324 GMAC** per 1080p frame, $\Delta\text{PSNR} \leq 0.1$ dB on 264 of 265 frame-rate pairs, bitstream byte for byte unchanged.

The guarantee costs nothing: choosing the multiplier per frame *raises* the mean saving, because one multiplier for a set solves the wrong problem.

The last improvement needed no training at all — 443,520 parameters re-solved in closed form, on the tiles that reach each exit.

What we cannot yet claim: wall-clock end to end, more than one decoder, and anything about inter frames.